줌줌투어 사내 온톨로지(지식그래프) 구축 가이드: 무료 솔루션 비교, 추천 구성, 단계별 로드맵

줌줌투어에는 **Neo4j Community Edition(무료, GPLv3) + 공식 Neo4j MCP 서버(읽기 전용 모드) + Git 저장소(원본 관리)**를 기본으로 쓰고, DB가 많아지면 **OpenMetadata(Apache 2.0, MCP 내장)**로 DB 스키마 자동 수집을 붙이기를 권합니다. 비전문가가 AI 에이전트와 함께 가장 빨리 결과를 내고 확장할 수 있는 조합입니다. RDF/OWL 표준 방식(Apache Jena Fuseki + Protégé)도 좋지만 학습 부담이 커서 2순위로 둡니다. 반드시 알아야 할 제약도 있습니다. Neo4j 무료판에는 권한(역할) 관리가 없어서 **팀원에게 DB 계정을 직접 주면 안 되고**, MCP 서버의 읽기 전용 모드와 Git 기반 변경 절차로 권한을 대신해야 합니다.

TL;DR

- **추천 조합**: Neo4j Community(그래프 DB) + 공식 Neo4j MCP 서버(팀원은 읽기 전용으로 접속) + Git 저장소(YAML/Cypher 파일이 원본이고 관리자만 반영). DB 스키마는 처음에는 AI 에이전트가 만든 스크립트로 `information_schema`에서 추출하고, DB·팀이 늘면 OpenMetadata(또는 DataHub)를 붙입니다. 모두 무료이고 사내 상업적 사용이 가능한 라이선스입니다.
- **대안 비교 요약**: RDF 표준을 원하면 Apache Jena Fuseki(Apache 2.0)+Protégé/WebProtégé, AI 메모리형 지식이 필요하면 Graphiti(Apache 2.0)를 씁니다. GraphDB Free는 오픈소스가 아니고 라이선스 신청과 동시 쿼리 제한이 있으며, Memgraph는 BSL 라이선스라서 1순위에서 뺐습니다.
- **성공 열쇠**: 처음부터 전사를 모델링하지 말고 "예약→결제→정산" 프로세스 하나로 6~8주 파일럿을 합니다. 클래스는 15개 안팎으로 시작하고, 원본은 Git에 두고, 운영 DB의 실제 데이터(개인정보)는 절대 넣지 않고 메타데이터(테이블·컬럼 구조·설명)만 넣습니다.

1. 먼저 알아야 할 핵심 용어 (비전문가용)

용어	쉬운 설명	줌줌투어 예시
온톨로지(Ontology)	회사 안의 "사물의 종류"와 "그 사이 관계"를 정한 **설계도/사전**	"시스템은 DB를 사용한다", "부서는 업무를 수행한다"는 규칙
지식그래프(Knowledge Graph)	온톨로지 설계도대로 실제 데이터를 채운 **점(노드)과 선(관계)의 지도**	`예약시스템 —USES→ booking_db`
노드(Node) / 클래스(Class)	점 하나 / 점의 종류	노드: "정산팀", 클래스: Department
관계(Relationship)	두 점을 잇는 화살표	`정산팀 —PERFORMS→ 공급사 정산`
속성 그래프(Property Graph)	노드·관계에 속성(이름, 담당자 등)을 붙이는 실용적 그래프 방식. 질의 언어는 **Cypher**	Neo4j, Memgraph
RDF/OWL	W3C 표준 그래프 방식. 모든 것을 "주어-술어-목적어" 3개 묶음(트리플)으로 표현하고 질의 언어는 **SPARQL**. 추론·표준 어휘 재사용에 강하지만 배우기 어려움	Fuseki, GraphDB, Protégé
메타데이터 카탈로그	회사 DB들의 테이블·컬럼·연결관계(리니지)를 **자동 수집**해 보여주는 도구	OpenMetadata, DataHub
MCP(Model Context Protocol)	AI 에이전트(Claude 등)가 외부 도구·DB에 접속하는 **표준 플러그**. 2025년 12월 Anthropic이 리눅스 재단 산하 Agentic AI Foundation에 기증함 Wikipedia	Claude Code가 MCP로 Neo4j를 조회
GraphRAG	그래프에서 관련 지식을 찾아 LLM 답변 근거로 쓰는 방식	"환불 처리에 어떤 테이블이 쓰이나?"에 그래프 근거로 답변

핵심 판단: 줌줌투어의 목적은 "학술적 추론"이 아니라 "시스템·DB·부서·업무의 연결을 팀이 AI로 조회·협업"하는 것입니다. 그래서 **속성 그래프(Neo4j)**가 RDF보다 실용적입니다. 표준 어휘(W3C ORG, PROV-O 등)는 개념만 빌려 쓰면 됩니다(6장). neo4j

2. 솔루션 조사 결과 (2025~2026년 기준)

2.1 그래프 저장소/온톨로지 엔진

① Neo4j Community Edition — 1순위 추천

- 라이선스: Community Edition은 GPLv3 오픈소스입니다. shortlisted neo4j 사내에서 수정 없이 쓰는 것은 자유롭고, 수정본을 외부에 배포할 때만 소스 공개 의무가 생깁니다. trustbeforeintelligence

- 버전: 2025.01부터 연-월 방식의 달력 버전(CalVer, YYYY.MM.Patch)을 씁니다. neo4j neo4j Docker Hub 공식 이미지에 2026.x 태그와 5.26 LTS 태그가 함께 있습니다. docker github 5.26 LTS는 2028년 11월까지 지원되고 4.4 LTS 지원은 2025년 11월 30일에 끝났습니다. neo4j
- 무료판 제약(중요): Neo4j 공식 문서는 Community Edition을 "단일 인스턴스 배포에 적합한 완전 기능판"으로 설명합니다. 클러스터링·온라인 백업·역할 기반 권한(RBAC)·다중 사용자 DB는 Enterprise 전용입니다. neo4j +3 공식 운영 매뉴얼에는 **"Community Edition에는 역할이 없고, 모든 사용자가 암묵적으로 관리자 권한을 가진다"**고 적혀 있습니다. neo4j 백업은 DB를 멈추고 하는 오프라인 백업만 됩니다. github
- AI 연동: Neo4j 공식 MCP 서버(github.com/neo4j/mcp, Go 바이너리)가 있습니다. 도구는 `get-schema`, `read-cypher`, `write-cypher`, `list-gds-procedures`이고, `NEO4J_READ_ONLY=true`(최신 버전은 `NEO4J_MCP_READ_ONLY`)를 켜면 쓰기 도구가 꺼집니다. github v1.1.0(2025-12-16)부터 HTTP 전송 모드를 지원하고, HTTP 모드에서는 요청마다 따로 인증합니다. github 다만 **Community Edition은 Bearer 토큰 인증을 지원하지 않아 Basic Auth를 써야 합니다.** neo4j 공식 서버 라이선스는 pkg.go.dev 기준 GPL-3.0이지만 Docker Hub에는 "Other"로 표시되어 있으니 LICENSE 파일을 직접 확인하세요. docker go 이와 별도로 Neo4j Labs의 `mcp-neo4j`(MIT)에는 Cypher 서버, 메모리 서버, **데이터 모델링 서버**가 있습니다. github repoportal 이 Labs 서버들은 Neo4j 제품팀의 공식 지원(SLA)을 받지 않습니다. mcpservers
- RDF 연계: neosemantics(n10s) 플러그인으로 OWL/RDFS/SKOS 온톨로지를 가져오고 내보낼 수 있고 SHACL 검증도 됩니다. neo4j 다만 최신 Neo4j 버전과 호환되기까지 시차가 생길 수 있습니다. Neo4j Online Community의 "Neosemantics in Neo4J Desktop" 스레드에서 Neo4j 5.27.0과 n10s 5.20.0을 쓰던 사용자는 "At the moment - n10s is not compatible with 5.27.0. You need to downgrade"라는 답변을 받았습니다.
- 장점: 생태계·튜토리얼·한국어 자료가 가장 많고, 브라우저 UI로 그래프를 바로 볼 수 있고, Cypher가 직관적이고, LLM의 Cypher 생성 품질이 높습니다. shortlisted
- 단점: 권한 관리·고가용성이 없고, OWL 추론이 안 됩니다. sw-expert flur

② Apache Jena Fuseki — RDF 표준 노선의 1순위

- 라이선스: Apache 2.0이라 제약이 없습니다. Wikipedia
- 버전: 5.5.0(2025-07), 5.6.0(2025-10), 6.0.0(2026-02). mvnrepository **CVE-2025-49656, CVE-2025-50151(관리자의 서버 외부 경로 파일 접근)이 5.4.0 이하에 있고 5.5.0에서 고쳐졌으므로** 5.5.0 이상을 쓰세요. apache
- 기능: SPARQL 1.1 질의·갱신 서버, 웹 UI, Apache Shiro 기반 로그인을 제공하고, Jena Permissions로 그래프 단위 권한을 줄 수 있습니다(직접 구현 필요). RDFS/OWL 추론기가 내장되어 있습니다. github +2
- AI 연동: 공식 MCP는 없고 커뮤니티 MCP 서버가 여럿 있습니다(`jena-mcp`, `mcp-jena`, 범용 `sparql-mcp` 등). github +2 품질과 유지보수 상태는 제각각입니다.
- 단점: SPARQL과 RDF를 배워야 해서 비전문가에게 진입장벽이 높고, 그래프 시각화가 약합니다.

③ Ontotext(Graphwise) GraphDB Free — 조건부

- 무료지만 **오픈소스가 아닙니다.** 11.0부터는 Free 에디션도 이메일로 무료 라이선스를 신청해 설치해야 하고, 마이너 버전을 올릴 때도 다시 신청해야 합니다. ontotext ontotext
- 동시 쿼리 제한이 있는데 공식 자료끼리 서로 다릅니다. GraphDB 11.5 Licensing 문서는 "GraphDB Free supports a limit of two concurrent queries"라고 하면서, 같은 문서에서 "each individual core supports only a single concurrent query"라고도 씁니다. Graphwise 제품 페이지는 "GraphDB Free comes with a single-core license"라고 적고 있어, 단일 코어 기준으로는 사실상 병렬 쿼리 1개로 읽힙니다. 또 11.5 문서는 라이선스 종류에 "a free-to-use non-commercial option"이 포함된다고 표현하므로, **사내 상업적 사용 가능 여부를 라이선스 원문으로 꼭 확인해야 합니다.**
- 장점: OWL 2 RL 추론, 시각화, "Talk to Your Graph"(GPT 연동), 관계형 DB 가상화 등 기능이 가장 풍부합니다. ontotext ontotext
- 판단: 무료 조건이 불확실하고 여러 팀원이 동시에 질의하는 용도에는 맞지 않아 줌줌투어에는 비추천입니다.

④ Protégé / WebProtégé — 온톨로지 "설계" 도구(저장소 아님)

- 스탠퍼드 대학이 만든 무료 오픈소스 OWL 2 편집기입니다. stanford 데스크톱판은 개인 설계용입니다.
- WebProtégé는 Docker(MongoDB 포함)로 사내에 설치할 수 있고, 변경 추적·이력, 공유·권한, 스레드 토론 등 협업 기능이 있습니다. 스탠퍼드 개발팀의 WWW 2019 논문(Horridge 외, "WebProtégé: A Cloud-Based Ontology Editor")에 따르면 공개 서비스는 "over 68,000 OWL ontology projects and over 50,000 user accounts"를 호스팅하고 있었습니다(2019년 기준 수치).
- 한계: 차세대 마이크로서비스 구조로 옮기는 중이라 레거시 저장소와 개발 저장소가 나뉘어 있습니다. github AI 에이전트(MCP) 연동이 없고, "클래스 설계" 위주라 수천 개의 테이블·컬럼 인스턴스를 관리하기에는 맞지 않습니다.

⑤ Oxigraph — 가벼운 RDF 저장소

- Rust로 만들었고 MIT/Apache-2.0 이중 라이선스입니다. docs gdb-engines SPARQL 1.1 서버이고 Docker 이미지 하나로 돌아갑니다(최신 CLI v0.5.x). crates github
- 인증 기능이 없어서 nginx로 Basic 인증을 앞에 붙여야 합니다. pypi 소규모 실험용으로 적합합니다.

⑥ TypeDB Community Edition — 강한 스키마형

- MPL 2.0 오픈소스이고 단일 서버로 돌아갑니다. 3.0은 2024년 12월 20일에 출시됐습니다. 공식 MCP 서버가 있고, 스키마에 어긋나는 쓰기와 질의는 오류로 거부하므로 AI가 엉뚱한 데이터를 넣는 일을 막는 데 유리합니다. typedb +2
- 단점: 고유 언어 TypeQL을 배워야 하고 한국어 자료와 생태계가 작습니다.

⑦ Memgraph Community — 비추천

- Cypher 호환에 빠르고 MCP 서버도 있습니다. 하지만 Business Source License(BSL) 1.1을 따르며, 라이선스 문서 스스로 "오픈소스 라이선스가 아니다"라고 밝힙니다. github mcp 사내 사용은 대체로 허용되지만, 완전 무료·오픈소스라는 요구에는 Neo4j CE가 더 깔끔합니다.

2.2 DB 스키마 자동 수집(메타데이터 카탈로그)

⑧ OpenMetadata — 카탈로그 1순위

- Apache 2.0 오픈소스입니다. OpenMetadata GitHub README에 따르면 “130+ connectors”(MySQL, PostgreSQL 등)로 테이블·컬럼·리니지·소유자·용어집을 자동으로 모읍니다.
- **MCP 서버가 내장**되어 있습니다(`{서버주소}/mcp`, Streamable HTTP). 사람은 OAuth 2.0로, 무인 에이전트는 Bot JWT 토큰으로 인증합니다. 제공 도구 수는 버전마다 다릅니다. v1.12.x “MCP Tools Reference”에는 search_metadata, semantic_search, get_entity_lineage, create_glossary_term, root_cause_analysis 등 16개가 있고, v2.0.x 연결 가이드에는 24개로 적혀 있으니 설치할 버전의 목록을 확인하세요.
- 사양: 로컬 Docker에는 최소 6GiB 메모리와 4 vCPU가 필요합니다. 운영 권장은 서버 4 vCPU/16GiB, DB 4 vCPU/16GiB, Elasticsearch/OpenSearch 2 vCPU/8GiB로 꽤 무겁습니다. open-metadata open-metadata

⑨ DataHub — 카탈로그 대안

- Apache 2.0이고, datahub-project/datahub README에 따르면 “150+ integrations”에서 메타데이터를 가져옵니다. DataHub 공식 Quickstart Guide가 `datahub docker quickstart`용으로 “Tested & confirmed config”라고 밝힌 구성은 2 CPU, 8GB RAM, 2GB 스왑, 13GB 디스크입니다.
- 오픈소스 MCP 서버(`mcp-server-datahub`)를 직접 띄울 수 있습니다. HTTP 모드에서는 사용자마다 자기 토큰을 쓰므로 **DataHub 권한과 감사 기록이 사용자별로 유지**됩니다(`METADATA_SERVICE_AUTH_ENABLED=true` 필요). github

2.3 LLM 연동형 지식그래프 도구

⑩ Graphiti(Zep) — 보조 도구

- Apache-2.0 오픈소스입니다. 회의록·문서·대화에서 개체와 관계를 자동으로 뽑아 **시간 정보가 있는 그래프**로 저장하고, 바뀐 사실은 이력으로 남깁니다. github getzep
- MCP 서버 1.0이 있고 백엔드로 Neo4j, FalkorDB, Neptune을 씁니다. 기본값인 FalkorDB는 SSPL 라이선스이므로 **Neo4j 백엔드를 고르는 것이 라이선스상 깔끔합니다.** getzep renezander
- LLM API 키가 필요하고(Ollama 같은 로컬 LLM도 가능), hexmos 자동 추출이라 정확성을 통제하기 어렵습니다. 그래서 "공식 온톨로지"가 아니라 업무 노하우를 기록하는 보조 계층으로 쓰는 것이 맞습니다.

3. 장단점 비교표

항목	Neo4j CE	Jena Fuseki	GraphDB Free	WebProtégé	OpenMetadata	DataHub	TypeDB CE	Graphiti
역할	그래프 DB	RDF 저장소	RDF 저장소	온톨로지 편집기	메타데이터 카탈로그	메타데이터 카탈로그	타입 그래프 DB	AI 메모리 그래프
데이터 모델	속성 그래프 (Cypher)	RDF/OWL(SPARQL)	RDF/OWL(SPARQL)	OWL 2	자체 메타데이터 그래프	자체 메타데이터 그래프	엔티티-관계 (TypeQL)	시간 속성 그래프
라이선스	GPLv3	Apache 2.0	독점 무료(신청 필요)	오픈소스(BSD 계열)	Apache 2.0	Apache 2.0	MPL 2.0	Apache 2.0
무료판 핵심 제약	RBAC·클러스터·온라인 백업 없음, 모든 사용자 관리자	없음	동시 쿼리 1~2개, 상업 조건 확인 필요	차세대 전환 중	운영 시 자원 많이 필요	운영 시 자원 많이 필요	단일 서버	LLM 비용/로컬 LLM 필요
비전문가 학습 난이도	하~중	상	상	중	하(UI 중심)	중	중~상	중
시각화/웹 UI	우수(Browser)	기본	우수	우수(편집 UI)	우수	우수	보통 (Studio)	약함
다중 사용자·권한	계정만 있고 역할 없음	Shiro+Permissions(구현 필요)	기본 보안	프로젝트 공유·권한	역할·팀·정책	역할·정책	기본 사용자	그룹 ID 분리
MCP 서버	공식 (Neo4j)+Labs	커뮤니티	커뮤니티(범용 SPARQL)	없음	**내장**	공식 오픈소스	공식	공식
DB 스키마 자동 수집	직접 스크립트	직접 스크립트	가상화 기능	불가	**130+ 커넥터**	**150+ 통합**	직접 스크립트	텍스트에서 추출
설치·운영	Docker 1개	Docker 1개	Docker 1개	Docker+MongoDB	Docker Compose(DB·검색엔진 포함)	Docker 다수 (Kafka 등)	Docker 1개	Docker+그래프DB
커뮤니티·한국어 자료	매우 많음	보통(학계)	보통	보통(학계)	많음	많음	적음	증가 중
줌줌 투어 적합도	★★★★★	★★★	★★	★★(설계 보조)	★★★★	★★★	★★★	★★(보조)

4. 줌줌투어 추천 구성

4.1 추천 아키텍처

4.2 이 구성을 고른 이유

1. **권한 문제를 구조로 해결합니다.** Neo4j CE에서는 모든 사용자가 관리자이므로 팀원은 DB에 직접 접속하지 않고 **읽기 전용 MCP 서버만** 씁니다. 수정은 Git Pull Request로 받고 관리자 검토 뒤 CI가 반영합니다. 무료판의 약점이 오히려 "검토된 지식만 반영"하는 좋은 운영 습관이 됩니다.
2. **AI 에이전트 친화성.** LLM은 Cypher를 SQL처럼 잘 생성하고, 공식 MCP의 `get-schema`가 그래프 구조를 에이전트에게 알려줍니다. github Labs의 데이터 모델링 MCP는 모델 설계와 적재 코드 생성을 도와줍니다. medium
3. **단계적 확장.** 처음에는 Neo4j 하나(서버 1대, Docker 1개)로 시작하고, DB 수가 많거나 스키마 변경이 잦아지면 OpenMetadata를 붙여 자동 수집합니다. OpenMetadata 자체 MCP를 함께 연결하면 “DB 상세는 카탈로그, 업무·부서·프로세스 연결은 Neo4j”로 역할이 나뉩니다.
4. **Git이 진짜 원본입니다.** 그래프 DB는 언제든 Git에서 다시 만들 수 있습니다. 덕분에 백업, 변경 이력, 누가 왜 바꿨는지 추적, 롤백이 무료로 해결되고, Community Edition에 온라인 백업이 없는 문제도 완화됩니다.

4.3 대안 시나리오

- **"표준 온톨로지(OWL)로 남기고 싶다"면:** Protégé(설계) → Turtle 파일을 Git에 저장 → Fuseki 5.5+에 적재 → 커뮤니티 SPARQL MCP로 연결합니다. 또는 n10s로 같은 OWL을 Neo4j에 가져오는 혼합 방식도 가능합니다(버전 호환을 확인하세요). github deepwiki
- **"회의록·위키 같은 비정형 노하우도 AI가 기억하게" 하려면:** Graphiti MCP를 같은 Neo4j(별도 그룹 ID)에 붙입니다. 공식 구조 그래프와 섞이지 않게 라벨이나 그룹을 나눠야 합니다.

5. 권장 온톨로지 구성(스키마 예시)

5.1 클래스(노드 라벨) — 처음에는 15개 안팎

영역	클래스	설명 / 주요 속성
IT 자산	`System`	서비스/애플리케이션(예: 예약시스템, 파트너어드민, 정산시스템). owner_team, url, status
	`Module`	시스템의 하위 기능/API(예: 결제모듈, 바우처발급)
	`Database`	DB 인스턴스/스키마. engine(MySQL 등), env(prod/stg)
	`Table`	테이블. description, pii(개인정보 포함 여부)
	`Column`	컬럼. data_type, is_pk, is_fk, pii_level
조직	`Department`	부서(상품기획팀, 운영팀, CS팀, 정산팀, 마케팅팀, 개발팀 등)
	`Role`	역할/직무(정산 담당자, CS 상담원, 상품 MD)
	`Person`	담당자(선택. 개인정보 최소화를 위해 사번이나 별칭만)
업무	`BusinessFunction`	상시 업무 기능(상품 등록, 예약 관리, 환불 처리, 공급사 정산)
	`Process`	순서가 있는 업무 흐름(예약→확정→바우처 발송)
	`Step`	프로세스 단계. order, actor_role, sla
도메인	`BusinessEntity`	업무 개념(상품, 예약, 결제, 고객, 파트너/공급사, 정산, CS 문의). 업무 용어와 테이블을 잇는 다리
문서	`Document`	매뉴얼, 정책, 위키, BPMN 파일 링크
용어	`Term`	사내 용어집(예: "확정대기", "노쇼")

5.2 관계(관계 타입)

관계	의미
`(System)-[:HAS_MODULE]->(Module)`	시스템 구성
`(System)-[:USES]->(Database)`	시스템이 DB 사용(읽기/쓰기 속성)
`(Table)-[:BELONGS_TO]->(Database)`	테이블 소속
`(Column)-[:BELONGS_TO]->(Table)`	컬럼 소속
`(Column)-[:REFERENCES]->(Column)`	FK 관계(자동 추출)
`(System)-[:INTEGRATES_WITH]->(System)`	시스템 간 연동(API, 배치, 외부 PG·공급사 API)
`(Department)-[:PERFORMS]->(BusinessFunction)`	부서의 업무
`(Department)-[:OWNS]->(System)`	시스템 책임 부서
`(Role)-[:MEMBER_OF]->(Department)`	역할 소속
`(BusinessFunction)-[:USES]->(System)`	업무에 쓰는 시스템
`(BusinessFunction)-[:REALIZED_BY]->(Process)`	업무를 구현하는 프로세스
`(Process)-[:HAS_STEP]->(Step)`	단계 포함
`(Step)-[:NEXT]->(Step)`	단계 순서
`(Step)-[:PERFORMED_BY]->(Role)`	단계 수행자
`(Step)-[:USES]->(System)`	단계에서 쓰는 화면/시스템
`(Step)-[:READS]->(Table)` / `[:WRITES]->(Table)`	단계의 데이터 읽기/쓰기
`(BusinessEntity)-[:STORED_IN]->(Table)`	업무 개념의 저장 위치
`(*)-[:DOCUMENTED_BY]->(Document)`	근거 문서

5.3 여행사 도메인 예시 (Cypher)

```cypher
```

```
// 제약조건: 이름 중복 방지 (Community Edition에서도 사용 가능한 고유성 제약)
CREATE CONSTRAINT system_name IF NOT EXISTS FOR (s:System) REQUIRE s.name IS UNIQUE;
CREATE CONSTRAINT table_fqn  IF NOT EXISTS FOR (t:Table)  REQUIRE t.fqn  IS UNIQUE;

// 시스템·DB·테이블
MERGE (bk:System {name:'예약시스템', status:'운영'})
MERGE (st:System {name:'정산시스템'})
MERGE (pg:System {name:'외부PG', external:true})
MERGE (db:Database {name:'booking_db', engine:'MySQL', env:'prod'})
MERGE (tRes:Table {fqn:'booking_db.reservation', description:'예약 마스터', pii:true})
MERGE (tPay:Table {fqn:'booking_db.payment', description:'결제 내역', pii:false})
MERGE (tSet:Table {fqn:'booking_db.supplier_settlement', description:'공급사 정산'})
MERGE (bk)-[:USES {mode:'read_write'}]->(db)
MERGE (st)-[:USES {mode:'read'}]->(db)
MERGE (bk)-[:INTEGRATES_WITH {via:'API'}]->(pg)
FOREACH (t IN [tRes,tPay,tSet] | MERGE (t)-[:BELONGS_TO]->(db))

// 부서·업무·프로세스
MERGE (ops:Department {name:'운영팀'})
MERGE (fin:Department {name:'정산팀'})
MERGE (cs:Department {name:'CS팀'})
MERGE (f1:BusinessFunction {name:'예약 확정 처리'})
MERGE (f2:BusinessFunction {name:'공급사 월 정산'})
MERGE (ops)-[:PERFORMS]->(f1)
MERGE (fin)-[:PERFORMS]->(f2)
MERGE (f1)-[:USES]->(bk)

MERGE (p:Process {name:'예약-결제-확정'})
MERGE (f1)-[:REALIZED_BY]->(p)
MERGE (s1:Step {name:'고객 결제 완료', order:1})
MERGE (s2:Step {name:'공급사 재고 확인', order:2})
MERGE (s3:Step {name:'예약 확정 및 바우처 발송', order:3})
MERGE (p)-[:HAS_STEP]->(s1) MERGE (p)-[:HAS_STEP]->(s2) MERGE (p)-[:HAS_STEP]->(s3)
MERGE (s1)-[:NEXT]->(s2) MERGE (s2)-[:NEXT]->(s3)
MERGE (s1)-[:WRITES]->(tPay)
MERGE (s3)-[:READS]->(tRes) MERGE (s3)-[:WRITES]->(tRes)
MERGE (s3)-[:PERFORMED_BY]->(:Role {name:'운영 담당자'});
```

팀원이 AI 에이전트에게 이렇게 물으면 에이전트가 MCP로 아래 Cypher를 만들어 실행합니다.

- "예약 테이블 구조를 바꾸면 영향받는 부서는?"

cypher

```
MATCH (t:Table {fqn:'booking_db.reservation'})<-[:READS|WRITES]-(s:Step)<-[:HAS_STEP]-(p:Process)
      <-[:REALIZED_BY]-(f:BusinessFunction)<-[:PERFORMS]-(d:Department)
RETURN DISTINCT d.name AS 부서, f.name AS 업무, p.name AS 프로세스, s.name AS 단계;
```

5.4 같은 내용의 Turtle(RDF) 예시 — 표준 어휘 재사용

turtle

```
@prefix zz:   <https://ontology.zoomzoomtour.internal/def#> .
@prefix org:  <http://www.w3.org/ns/org#> .
@prefix prov: <http://www.w3.org/ns/prov#> .
@prefix dcat: <http://www.w3.org/ns/dcat#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

zz:Department   rdfs:subClassOf org:OrganizationalUnit .
zz:Step         rdfs:subClassOf prov:Activity .
zz:Table        rdfs:subClassOf dcat:Dataset .

zz:FinanceTeam  a zz:Department ; rdfs:label "정산팀"@ko ; org:unitOf zz:ZoomZoomTour .
zz:Step_Confirm a zz:Step ; rdfs:label "예약 확정 및 바우처 발송"@ko ;
                prov:used zz:Table_reservation .
zz:Table_reservation a zz:Table ; rdfs:label "booking_db.reservation" .
```

6. 재사용할 표준 온톨로지/어휘

표준	네임스페이스	무엇에 쓰나	줌줌투어 적용
W3C ORG (Organization Ontology)	`http://www.w3.org/ns/org#`	조직·하위조직·역할·소속 (org:Organization, OrganizationalUnit, Role, Membership, Post, Site 등 9개 클래스)	Department/Role/MEMBER_OF 설계 근거. 겸직·기간 정보는 Membership 패턴 차용 w3 dublincore
W3C PROV-O	`http://www.w3.org/ns/prov#`	활동(prov:Activity)이 엔티티를 사용(prov:used)하고 생성(prov:wasGeneratedBy)하는 출처·이력 모델 casrai github	Step의 READS/WRITES, "누가 언제 이 정보를 바꿨나" 기록
W3C DCAT 3 (2024 권고안)	`http://www.w3.org/ns/dcat#`	데이터 카탈로그·데이터셋·데이터 서비스 기술 dublincore govdataguide	Database=카탈로그, Table=데이터셋 개념 정렬
BBO (BPMN 2.0 Based Ontology)	`http://BPMNbasedOntology#`	BPMN 2.0의 프로세스·태스크·게이트웨이 개념을 OWL로 표현 (GPLv3) github nih	Process/Step 설계 참고. 실행 이력은 범위 밖이라 PROV-O와 병행 arxiv
schema.org	`https://schema.org/`	여행 상품·예약·조직의 일반 어휘 (TouristTrip, Reservation, Offer 등)	BusinessEntity(상품·예약·결제) 이름과 속성 참고
SKOS	`http://www.w3.org/2004/02/skos/core#`	용어집·동의어·상하위어	사내 용어집(Term)

판단: Neo4j 노선이라면 이 표준들을 "그대로 임포트"하지 말고, **이름과 구조를 빌려 쓰는 정도**로 충분합니다. 노드에 `standard_ref:'org:OrganizationalUnit'` 같은 속성을 달아 두면, 나중에 RDF로 내보낼 때(n10s) 쉽게 대응시킬 수 있습니다. github

BPMN 연계: 프로세스 도면은 무료 오픈소스 BPMN 편집기(예: bpmn.io 계열, Camunda Modeler)로 그려 `.bpmn` XML을 Git에 저장합니다. AI 에이전트가 XML의 task, sequenceFlow, lane을 읽어 `Process/Step/NEXT/PERFORMED_BY`로 바꾸는 스크립트를 만들게 하면 도면과 그래프가 함께 유지됩니다.

7. 단계별 구축 로드맵 (파일럿 6~8주 + 확장)

단계	기간	할 일	AI 에이전트 활용	산출물
0. 범위 정의	1주	"이 그래프로 답하고 싶은 질문" 20개 수집(예: "환불 시 어느 테이블이 바뀌나?", "정산팀이 쓰는 시스템은?"). 파일럿 범위를 예약→결제→확정→정산 1개 흐름으로 한정	부서 인터뷰 메모를 주고 "질문 목록과 필요한 개념"을 정리시킴	질문 목록(역량 질문), 범위 문서
1. 개념(클래스) 도출	1주	5.1의 클래스 초안을 질문 목록에 맞게 다듬기. 15개 안팎 유지	질문 목록→클래스·속성 후보 제안, 중복·모호 지적	`ontology/schema.yaml`
2. 관계 정의	0.5주	5.2 관계 확정, 이름 규칙(대문자 동사형) 고정	Labs 데이터 모델링 MCP로 모델 검증, Cypher 제약조건 생성	`schema.cypher` (제약·인덱스)
3. 인프라 설치	0.5주	Docker로 Neo4j CE + 공식 MCP(HTTP, 읽기 전용) + nginx 구성, Git 저장소 생성	docker-compose.yml, nginx 설정 초안 작성	사내 서버 가동
4. DB 스키마 자동 추출	1주	운영 DB에 **읽기 전용·메타데이터 전용 계정**을 만들고 `information_schema`에서 테이블·컬럼·FK만 추출해 YAML로 생성	추출 스크립트(Python) 작성, 테이블 설명이 없으면 이름으로 설명 초안 생성(사람이 검수)	`ontology/db/*.yaml`
5. 부서·업무·프로세스 모델링	2주	부서별 1시간 워크숍으로 업무 기능과 프로세스 단계 확정. 필요하면 BPMN 도면 작성	업무 매뉴얼·위키를 읽혀 Process/Step 초안 생성, 담당 부서가 검수	`ontology/process/*.yaml`, `.bpmn`
6. 데이터 적재	0.5주	YAML→Cypher MERGE 변환 스크립트 작성, Git push 시 자동 적재(CI)	변환·적재 스크립트와 검증 쿼리(고아 노드, 누락 관계) 작성	CI 파이프라인
7. AI 에이전트 연결	0.5주	팀원 PC의 Claude Code/Desktop에 읽기 전용 MCP 등록, 사용 가이드 배포	팀원용 "질문 예시 모음"과 프롬프트 템플릿 작성	연결 가이드
8. 협업·권한 운영	상시	팀원 수정 요청 = AI가 YAML 수정 PR 작성 → 부서 담당자와 온톨로지 관리자 승인 → CI 반영	PR 초안 작성, 변경 영향 분석	운영 규칙
9. 유지보수	상시	주 1회 스키마 재추출 후 차이(diff)를 PR로, 분기별 클래스 정리	스키마 변경 차이 요약, 끊어진 연결 탐지	변경 이력
10. 확장(선택)	3개월 이후	DB 10개 이상이거나 스키마 변경이 잦으면 OpenMetadata를 도입해 자동 수집과 리니지 확보, 그 MCP도 연결	커넥터 설정, OpenMetadata→Neo4j 동기화 스크립트	카탈로그 통합

MCP 연결 예시(팀원 PC, 읽기 전용): 서버 쪽에서 공식 Neo4j MCP를 HTTP 모드와 `NEO4J_MCP_READ_ONLY=true`로 띄우고, 팀원은 `claude mcp add --transport http zz-ontology https://ontology.사내도메인/mcp` 형태로 등록합니다. Community Edition은 Basic Auth만 지원하므로 Neo4j에 팀원별 계정을 만들어 요청 헤더로 보내게 합니다. neo4j deepwiki 이렇게 하면 계정별 접속 기록을 남길 수 있지만, 권한을 실제로 막는 것은 읽기 전용 MCP와 네트워크 통제입니다. 버전에 따라 환경변수 이름이 다르니(`NEO4J_READ_ONLY` → `NEO4J_MCP_READ_ONLY`) 설치한 버전의 README를 확인하세요. github go

Git 저장소 구조 예시

```
zz-ontology/
 ├─ ontology/schema.yaml          # 클래스·관계 정의(설계도)
 ├─ ontology/db/booking_db.yaml # 자동 추출 + 사람이 단 설명
 ├─ ontology/org/departments.yaml
 ├─ ontology/process/reservation_confirm.yaml (+ .bpmn)
 ├─ scripts/extract_schema.py, load_to_neo4j.py, validate.cypher
 └─ docs/questions.md              # 역량 질문 = 회귀 테스트
```

8. 사내 서버 권장 사양

구성	권장 사양	근거
1단계(Neo4j + MCP + nginx)	4 vCPU / 16GB RAM / SSD 200GB	메타데이터만 넣으면 수십만 노드 이하라 충분함(추정치)
2단계 (+OpenMetadata)	8 vCPU / 32GB RAM / SSD 500GB 이상(별도 서버 권장)	OpenMetadata 운영 권장이 서버 4 vCPU/16GiB + DB 4 vCPU/16GiB + 검색엔진 2 vCPU/8GiB이고, 로컬 Docker 최소가 6GiB/4 vCPU open-metadata open-metadata
DataHub로 대체 시	퀵스타트 검증 구성 2 CPU/8GB RAM/2GB 스왑/13GB 디스크, 운영은 16~32GB	DataHub 공식 Quickstart Guide의 “Tested & confirmed config”

OS는 Ubuntu LTS + Docker Compose를 쓰고, 매일 밤 Neo4j 덤프를 받습니다. CE는 오프라인 백업만 되므로 짧은 중지 시간을 감수하거나, Git에서 다시 만드는 방식으로 복구합니다.

9. 보안 고려사항

1. **실데이터 금지 원칙:** 그래프에는 테이블·컬럼 구조와 설명(메타데이터)만 넣습니다. 예약자 이름·연락처·결제정보 같은 **실제 행 데이터는 절대 넣지 않습니다.** 컬럼에 `pii_level` 속성만 표시합니다.
2. **원천 DB 접근 최소화:** 스키마 추출 계정은 `information_schema` 조회 권한만 주고, 운영 DB 복제본이나 스테이징을 우선 씁니다.
3. **Neo4j CE의 권한 부재 보완:** 팀원에게 Bolt(7687)·브라우저(7474) 포트를 열지 않고 방화벽으로 관리자 IP만 허용합니다. 팀원 경로는 읽기 전용 MCP 하나뿐입니다. 공식 MCP의 `read-cypher`는 쿼리 유형 분류로 읽기 전용을 강제하지만, "잘못 분류된 사용자 정의 프로시저는 우회할 수 있다"는 단서가 README에 있으므로 github 불필요한 플러그인은 설치하지 않습니다.
4. **MCP 서버 노출 통제:** MCP 서버는 기본적으로 127.0.0.1에 묶고 neo4j github nginx(HTTPS + 사내 SSO/VPN)를 통해서만 열며, CORS·허용 호스트 설정을 최소화합니다.
5. **외부 LLM으로 나가는 정보:** 팀원이 클라우드 LLM(Claude 등)을 쓰면 조회 결과(시스템 구조·테이블명)가 LLM 공급자에게 전송됩니다. 회사의 AI 사용 정책·계약(데이터 학습 미사용 조건 등)을 확인하고, 민감한 시스템(결제 키 관리 등)은 그래프에 `confidential` 라벨을 붙여 읽기 전용 MCP 결과에서 빼는 별도 DB로 분리하는 것도 고려하세요.
6. **비밀정보 금지:** 접속 문자열, 비밀번호, API 키는 그래프와 Git 어디에도 넣지 않습니다(CI에서 시크릿 스캐너 적용).
7. **취약점 관리:** Fuseki를 쓴다면 5.5.0 이상, apache Neo4j·OpenMetadata는 분기마다 업데이트합니다.

10. 흔한 실패 요인과 피하는 법

실패 요인	증상	예방책
처음부터 전사 모델링	몇 달째 설계만 하고 아무도 안 씀	프로세스 1개 파일럿, 역량 질문 20개에 답하는 것을 완료 기준으로
클래스 과다·추상화 과잉	AI도 사람도 헷갈림	15개 안팎 유지, 새 클래스는 "이걸로 답할 새 질문"이 있을 때만
AI 자동 생성 결과를 검수 없이 반영	그럴듯하지만 틀린 관계가 쌓여 신뢰 붕괴	AI는 초안과 PR까지만, 부서 담당자 승인 필수. `source`, `verified_by` 속성 기록
원본이 그래프 DB 안에만 있음	누가 바꿨는지 모름, 복구 불가	Git이 원본이고 그래프는 결과물
주인 없는 온톨로지	6개월 뒤 낡은 정보	부서별 "데이터 스튜어드" 지정, 스키마 자동 재추출과 변경 diff PR
이름 불일치	"예약/부킹/reservation"이 따로 노드가 됨	고유성 제약과 `Term` 용어집, 동의어 속성(aliases)
도구 먼저 고르기	RDF·추론 등 필요 없는 복잡성	질문 목록이 먼저, 도구는 그 다음
권한 착각	Neo4j CE에 팀원 계정을 주고 "읽기만 하라"고 안내	CE는 모든 사용자가 관리자이므로 구조적으로 차단(읽기 전용 MCP + 방화벽)

11. 최종 권고

1. **이번 주:** 질문 20개 수집, 파일럿 범위(예약→결제→확정→정산) 확정, 온톨로지 관리자 1명과 부서별 스튜어드 지정.
2. **1~2주차:** 사내 서버에 Neo4j CE(5.26 LTS 또는 최신 2026.x) + 공식 Neo4j MCP(읽기 전용, HTTP) + nginx를 Docker로 구성하고 Git 저장소를 만듭니다. AI 에이전트(Claude Code)에게 docker-compose, 스키마 추출 스크립트, 적재 CI를 만들게 합니다.
3. **3~6주차:** 5장의 스키마로 파일럿 데이터를 적재하고 팀원 5~10명에게 읽기 전용 MCP로 공개합니다. 역량 질문에 대한 답변 정확도를 측정합니다.
4. **3개월 후 판단:** DB·시스템 수가 많아 수동 관리가 버거우면 OpenMetadata를 별도 서버에 도입합니다. 표준 RDF 산출물이 필요해지면 n10s나 Fuseki 병행을 검토합니다.

Caveats

- 라이선스 해석: Neo4j CE(GPLv3)는 사내 사용에 문제가 없지만, 수정본을 외부에 배포하거나 서비스로 제공하는 경우의 의무는 법무 검토가 필요합니다. trustbeforeintelligence 공식 Neo4j MCP 서버의 라이선스는 출처마다 표기가 다르므로(pkg.go.dev: GPL-3.0, Docker Hub: Other) 저장소의 LICENSE 파일을 직접 확인하세요. docker go GraphDB Free의 상업적 사용 조건과 동시 쿼리 한도도 공식 자료끼리 다르게 적혀 있습니다. ontotext ontotext
- 버전과 설정 이름: Neo4j MCP 서버는 빠르게 바뀌고 있어(환경변수 접두어 변경, v2의 다중 테넌트 HTTP 모드 등) neo4j 이 문서의 설정 예시는 설치 시점의 README를 기준으로 다시 확인해야 합니다. n10s는 최신 Neo4j와 호환되기까지 시차가 있을 수 있습니다. neo4j
- 서버 사양 중 1단계 수치는 메타데이터 규모를 가정한 추정치이고, OpenMetadata와 DataHub 수치는 공식 문서 기준입니다.
- 커뮤니티 MCP 서버(Fuseki용 등)는 개인이나 소규모 팀이 유지보수하므로 운영에 쓰기 전에 코드와 보안을 검토해야 합니다.
- 한국어 자료가 많은지에 대한 평가는 일반적인 생태계 규모에 근거한 정성 판단입니다.